Google Summer of Code 2026 - Project Proposal

Expand kornia-vlm with ONNX and TensorRT Backends

Portable High-Performance VLM Inference in Rust from Desktop to Jetson Edge Devices

---

Full Name	Kiran Dharmar
Email	kirandharmar867@gmail.com
GitHub	github.com/kirandharmar867-maker
LinkedIn	linkedin.com/in/kiran-dharmar-73718437a
Organization	Kornia
Project	Expand kornia-vlm with ONNX and TensorRT Backends
Mentors	Edgar Riba, Miquel Farre
College	Govt Polytechnic Vijayapura
Year of Study	2024-2027
Country	India
Timezone	IST - Asia/Kolkata (UTC+5:30)
Duration	350 hours | Difficulty: Medium-Hard
GSoC Timeline	Community Bonding: Apr 30 - May 25 | Coding: May 26 - Aug 18

AI Tooling Disclosure (per kornia-rs policy): *AI assistance was used for research, grammar review, and proposal structure. Every technical claim, design decision, code snippet, and implementation plan was authored, verified, and is fully understood by me. I am 100% responsible for correctness and will disclose AI-assisted code in future PRs.*

---

1. Abstract

kornia-rs is a high-performance 3D computer vision library written in Rust, designed to be memory-safe, thread-safe, and suitable for real-time applications. Its kornia-vlm crate currently uses Candle for VLM inference, a framework that is powerful for training but heavyweight for production deployment, especially on edge hardware such as the NVIDIA Jetson. This project delivers a modular, production-grade inference backend for kornia-vlm built on ONNX Runtime, with an optional TensorRT execution path for CUDA-capable devices. The deliverables are: 1 to 2 VLMs fully integrated into kornia-vlm with CPU, CUDA, and TensorRT runtimes selectable at build time via Cargo feature flags; comprehensive benchmarks comparing PyTorch, ONNX, and TensorRT across latency, throughput, and memory; a working demo targeting Jetson Orin or equivalent cloud GPU; and full documentation. All work will be contributed upstream to kornia-rs.

2. Motivation

The frontier of computer vision is moving fast toward vision-language models. kornia-rs has already taken bold steps: SmolVLM2 landed in v0.1.10, and the community opened issue #634 specifically requesting an ONNX-based inference path. The demand is clear and the direction is set, but the critical infrastructure, a portable and optimized inference backend, is still missing.

Candle is excellent for training-focused workflows but carries unnecessary weight for inference-only deployments. Teams deploying VLMs on Jetson devices, factory floor cameras, or embedded robotics systems need lean, predictable, hardware-accelerated inference. ONNX Runtime provides exactly that: a battle-tested, cross-platform inference engine with a TensorRT backend that delivers best-in-class GPU throughput on NVIDIA hardware.

My existing Kornia contributions, including the LightGlue edge-case fix (PR #3565) and the FaceDetector robustness improvements (PR #3520), have given me a solid understanding of how kornia-rs is built and what quality standards it demands. This project is the natural next step: building something foundational and leaving kornia-vlm dramatically more capable for the entire community.

3. About Me and Qualifications

I am Kiran Dharmar, a CS student at Govt Polytechnic Vijayapura(2024-2027), India. I have been an active open-source contributor for over a year withmerged pull requests across Kornia, FOSSASIA/Visdom, and Biopython. My technicalbackground spans PyTorch model workflows, ONNX export, inference pipelines, data engineering, and basic Rust. I am actively deepening my Rust skills specifically for this project through the ort crate documentation and kornia-rs source study.

I am already in contact with Edgar Riba and Miquel Farre via kornia Discord. I have read CONTRIBUTING.md and AI_POLICY.md carefully and understand the Sole Responsible Author requirement, the pre-discussion rule, and the proof-of-verification standard for every PR.

4. Open Source Contributions

4.1 Kornia (Primary Organization)

PR #3565 (Merged): Fixed a runtime crash in the LightGlue feature matcher caused by empty descriptor tensors after keypoint pruning. Added a guard that handles the degenerate case gracefully instead of raising an IndexError mid-inference. This required understanding the full matching pipeline, the tensor shapes at each stage, and the conditions under which pruning produces empty outputs. One of my most technically demanding contributions.

PR #3614 (Pending): Improved the Quaternion constructor with strict input validation: rejects scalar tensors, enforces last dimension equals four, raises clear early errors. Defensive API design like this is especially important in robotics and 3D vision pipelines

where silent incorrect inputs cause hard-to-debug failures downstream.

PR #3520 (Merged): Improved the FaceDetector module with missing exception handling and additional tests including coverage for torch.compile (Dynamo) execution paths. Demonstrates my commitment to test quality and modern PyTorch compatibility, both directly relevant to ONNX export validation in this project.

4.2 FOSSASIA / Visdom

PR #959 (Merged): Fixed wildcard routing bug that intercepted static CSS and JavaScript asset requests. PR #958 (In Progress): Extended heatmap with custom axis value support. Received valuable maintainer feedback about API design, which I apply to all subsequent work.

4.3 Biopython

PR #5133 (Merged): Modified PSEA execution to use a temporary directory for clean, reproducible runs. PR #5157 (Pending): Fixed PDBList URL construction for non-RCSB mirrors. PR #5158 (In Progress): Documented thread-safety behaviour of PDBParser to prevent concurrency bugs.

5. Technical Implementation Details

5.1 Current Limitation: Candle-Only Inference

The kornia-vlm crate currently uses Candle as its sole inference backend. Candle is well-designed for training but carries significant overhead for inference-only deployments: heavy compile times, no native TensorRT path, and no support for ONNX Runtime's hardware-specific optimizations. Community issue #634 explicitly requested an ONNX-based path. SmolVLM2 was integrated in v0.1.10 as the first VLM. This project builds on that foundation by adding a proper multi-backend inference layer.

5.2 Full Inference Pipeline: PyTorch to TensorRT

The complete pipeline from a trained PyTorch VLM to an optimized TensorRT engine in kornia-rs involves four clearly defined stages:

Stage 1: PyTorch to ONNX Export

The model is exported from PyTorch using torch.onnx.export with dynamic axes to support variable input sizes. The preprocessing graph (resize, normalize, tokenize) is exported alongside the model graph so that the full inference pipeline, not just the model weights, is captured in the ONNX file.

```
# Export SmolVLM2 encoder + preprocessing to ONNX
torch.onnx.export(
model,
dummy_input,
"smolvlm2.onnx",
opset_version=17,
input_names=["pixel_values", "input_ids"],
output_names=["logits"],
```

```
dynamic_axes={"pixel_values": {0: "batch"}, "logits": {0: "batch"}}
)
```

Stage 2: ONNX Validation

The exported ONNX graph is validated using onnxruntime in Python to confirm numerical parity with the original PyTorch output. Tolerance threshold is 1e-4. This step catches export errors such as unsupported ops or incorrect dynamic axis handling before they reach the Rust integration layer.

```
import onnxruntime as ort
sess = ort.InferenceSession("smolvlm2.onnx")
ort_out = sess.run(None, {"pixel_values": img_np, "input_ids": ids_np})
assert np.allclose(torch_out, ort_out[0], atol=1e-4), "Parity check failed"
```

Stage 3: TensorRT Engine Build (optional, CUDA only)

On CUDA-capable hardware, the validated ONNX file is compiled into a TensorRT engine using trtexec or the TensorRT Python API. The engine is serialized to a .plan file for warm-start inference. FP16 precision is applied where supported by the hardware.

```
# Using trtexec (command line)
trtexec --onnx=smolvlm2.onnx \
--saveEngine=smolvlm2_fp16.plan \
--fp16 --workspace=2048

# Or via TensorRT Python API for programmatic control
builder = trt.Builder(logger)
network = builder.create_network(flags)
parser = trt.OnnxParser(network, logger)
parser.parse_from_file("smolvlm2.onnx")
engine = builder.build_serialized_network(network, config)
```

Stage 4: Integration into kornia-vlm Rust API

The ONNX model and TensorRT engine are loaded in Rust using the ort crate (ONNX Runtime Rust bindings). A clean InferenceBackend trait abstracts the backend selection so that the calling code is identical regardless of whether CPU, CUDA, or TensorRT is in use. The backend is selected at compile time via Cargo feature flags.

```
// InferenceBackend trait in kornia-vlm
pub trait InferenceBackend: Send + Sync {
fn infer(&self;, image: &Image;) -> Result;
}

// OrtBackend: selects CPU or CUDA execution provider
pub struct OrtBackend {
session: ort::Session,
}

impl OrtBackend {
pub fn new(model_path: &str;, provider: ExecutionProvider)
-> Result {
let session = Session::builder()?
.with_execution_providers([provider])?
.commit_from_file(model_path)?;
Ok(Self { session })
```

```
}
}
```

Pipeline Summary: PyTorch model -> torch.onnx.export -> ONNX file -> onnxruntime parity check -> trtexec / TensorRT API (GPU only) -> serialized .plan engine -> ort crate in Rust -> kornia-vlm InferenceBackend -> unified kornia-vlm public API

5.3 Runtime Selection via Cargo Feature Flags

Users select the backend at build time with no runtime configuration changes. This follows kornia-rs's existing pattern of modular crate features and keeps the binary minimal for deployment.

```
# CPU only (default, works everywhere)
kornia-vlm = { version = "0.1" }

# CUDA acceleration
kornia-vlm = { version = "0.1", features = ["onnx-cuda"] }

# TensorRT for maximum Jetson performance
kornia-vlm = { version = "0.1", features = ["onnx-tensorrt"] }
```

5.4 Model Selection

Primary model: SmolVLM2-256M (already in kornia-rs via Candle). Porting to ONNX validates parity immediately and gives a direct benchmark comparison. Second model to be confirmed with mentors during community bonding, selected from MoonDream2 or PaliGemma-3B based on ONNX exportability, license compatibility, and Jetson memory constraints (target under 4GB VRAM).

5.5 Backend Comparison

Feature	Candle (current)	ONNX Runtime	TensorRT
Inference speed	Moderate	Fast	Fastest on GPU
Edge deployment	Heavy	Lightweight	Optimized
FP16 and INT8	Limited	Supported	Full calibration
Cross-platform	Good	Excellent	NVIDIA only
Compile time	High	Low	Low + engine build
Export tool	N/A	torch.onnx	trtexec / TRT API
Rust binding	candle-core	ort crate	ort crate (TRT EP)

6. Project Goals and Deliverables

All deliverables are upstream contributions to kornia-rs. Every milestone produces a merged or review-ready pull request.

- **Goal 1 - ONNX Export Pipeline:** torch.onnx.export scripts for SmolVLM2 and second model, with onnxruntime parity validation at 1e-4 tolerance.

- **Goal 2 - OrtBackend CPU and CUDA:** Full InferenceBackend implementation using ort crate. CPU default; CUDA via feature flag. No unwrap(), idiomatic Rust, full Rustdoc.
- **Goal 3 - TensorRT Execution Path:** TrtBackend using ORT TensorRT Execution Provider. trtexec-based engine build, .plan serialization for warm-start, FP16 support. Tested on Jetson Orin or cloud T4.
- **Goal 4 - Benchmarking Suite:** Criterion benchmarks comparing PyTorch vs ONNX vs TensorRT: latency (ms/image), throughput (FPS), memory (MB). Results committed as artifacts.
- **Goal 5 - kornia-vlm API Integration:** Both models accessible via unified public API. Backward-compatible. Example in examples/ directory.
- **Goal 6 - CLI Demo Application:** Working demo with --model, --backend, --image flags. Reproducible from README. Demo video recorded.
- **Goal 7 - Documentation:** API docs for all public types, README updates, Jetson setup guide covering ORT install and TRT engine build steps.

7. Detailed Week-by-Week Implementation Plan

The GSoC 2026 coding period runs May 26 to August 18 (12 weeks), preceded by Community Bonding April 30 to May 25. Each week has a specific, measurable output.

Community Bonding -- April 30 to May 25

Period	Output
Apr 30 - May 7	Deep review of kornia-vlm source, SmolVLM2 Candle integration, ort crate API study
May 8 - May 16	Confirm second VLM with mentor. Set up cross-compile environment for Jetson/cloud GPU
May 17 - May 25	Technical Design Document submitted to mentors. Warm-up PR to kornia-rs merged

Phase 1 -- ONNX Export and Trait Foundation

Week	Dates	Specific Output
Week 1	May 26 - Jun 1	InferenceBackend trait implemented with Rustdoc. Mock backend unit tests passing
Week 2	Jun 2 - Jun 8	SmolVLM2 exported to ONNX via torch.onnx.export. Preprocessing graph included
Week 3	Jun 9 - Jun 10	Parity validation: ORT vs Candle output within 1e-4. PR submitted for review

Phase 2 -- OrtBackend CPU and CUDA (Midterm)

Week	Dates	Specific Output
Week 4	Jun 11 - Jun 17	OrtBackend with CPU provider complete. End-to-end inference test passing on CI

Week 5	Jun 18 - Jun 24	CUDA execution provider added behind onnx-cuda feature flag. GPU tests passing
Week 6	Jun 25 - Jun 30	First Criterion benchmarks: CPU latency and throughput committed. Midterm PR open

Phase 3 -- Second VLM Integration

Week	Dates	Specific Output
Week 7	Jul 1 - Jul 7	Second VLM exported to ONNX. Parity validated. Preprocessing pipeline complete
Week 8	Jul 8 - Jul 18	Second VLM integrated into kornia-vlm API. CPU vs CUDA comparative benchmarks committed

Phase 4 -- TensorRT Backend and Edge Testing

Week	Dates	Specific Output
Week 9	Jul 19 - Jul 25	TrtBackend implemented. trtexec engine build working. .plan serialization complete
Week 10	Jul 26 - Aug 1	FP16 precision validated. Full benchmark suite: PyTorch vs ONNX vs TensorRT committed
Week 11	Aug 2 - Aug 8	Jetson / cloud GPU tests complete. Demo video recorded. All benchmarks finalized

Phase 5 -- Documentation and Final Submission

Week	Dates	Specific Output
Week 12	Aug 9 - Aug 18	CLI demo polished. All docs written. Final PR submitted. GSoC portal submission done

8. Test Plan

Unit tests will cover the InferenceBackend trait via mock backends; ONNX export parity validating ORT output versus Candle output within 1e-4; preprocessing pipeline output shape and value correctness; and error handling confirming all invalid inputs return Result::Err with no panics or unwrap() calls.

Integration tests will cover end-to-end inference on test images with expected output categories; CPU backend running on any CI machine with no GPU required; CUDA backend tested locally with test logs committed as PR artifacts per kornia-rs AI policy; and both models producing consistent outputs across repeated runs to verify determinism.

CI and CD: GitHub Actions runs all unit and CPU integration tests on every PR. Cargo clippy is enforced with no warnings permitted. cargo fmt enforces consistent formatting. All PRs include pasted cargo test output as proof of local execution.

9. Benchmarking Plan

A dedicated benchmarking suite will be implemented using the Criterion crate and committed to the kornia-rs repository as a permanent performance artifact. The goal is to give maintainers and users a clear, reproducible picture of the performance tradeoffs between each runtime configuration.

9.1 What Will Be Measured

Metric	Description	Tool
Latency (ms/image)	Time from raw image input to decoded text output, single image	Criterion
Throughput (FPS)	Images processed per second at batch sizes 1 and 4	Criterion
Memory usage (MB)	Peak RSS before and after session creation and inference	Custom probe
Warm vs cold start	First inference (engine load) vs subsequent (cached) latency	Criterion
FP32 vs FP16 accuracy	Output quality comparison when TensorRT FP16 is enabled	Manual eval

9.2 What Will Be Compared

Configuration	Hardware	Expected Result
PyTorch (Candle baseline)	CPU / GPU	Baseline reference for all comparisons
ONNX Runtime CPU	CPU	Faster than Candle CPU; portable
ONNX Runtime CUDA	NVIDIA GPU	Significant speedup over CPU baseline
TensorRT FP32	NVIDIA GPU	Faster than ONNX CUDA; same accuracy
TensorRT FP16	Jetson / T4	Best latency; verify accuracy within 2%

9.3 Sample Benchmark Output Format

Benchmark results will be committed to the repository in the following format so they are reproducible and reviewable by maintainers:

```
Model: SmolVLM2-256M | Input: 224x224 RGB | Device: Jetson Orin
---------------------------------------------------------------
Backend Latency (ms) Throughput (FPS) Memory (MB)
Candle CPU 142.3 7.0 480
ONNX CPU 89.1 11.2 310
ONNX CUDA 18.4 54.3 620
TensorRT FP32 12.7 78.7 580
TensorRT FP16 8.2 121.9 410
```

10. Risks and Mitigations

Five risks have been identified with specific technical mitigations prepared for each.

Risk	Likelihood	Impact	Mitigation Strategy
ONNX conversion failure (unsupported ops, dynamic shapes)	Medium	High	Validate exportability during bonding. Use opset 17. Simplify model graph if needed. Fallback: export only encoder, handle decoder separately.
Unsupported TensorRT operations (custom attention, RoPE embeddings)	Medium	Medium	Use TensorRT plugin API for unsupported ops. Fallback: keep TrtBackend as optional; ONNX CUDA backend is sufficient for most targets.
GPU or Jetson hardware unavailability	Low	Medium	Cloud GPU (AWS T4 or Colab Pro) agreed with mentors as equivalent. All benchmark comparisons will be clearly documented with hardware specs.
ort crate API changes between versions	Low	Low	Pin exact ORT version in Cargo.toml. TrtBackend is fully abstracted so updates only touch one struct, not the public API.
Scope overrun (350 hours)	Low	Medium	Goals 1-5 are non-negotiable core. Goals 6-7 are enhancements. Week-by-week milestones are sized conservatively with buffer days built in.

11. Community Engagement and Post-GSoC Plans

During the coding period I will post weekly written progress updates in the kornia Discord GSoC channel. All PRs will include detailed descriptions, local test logs, and benchmark results. Every design decision will be documented in PR comments so future contributors can understand the reasoning. I will actively help other contributors in adjacent areas of kornia-rs.

I plan to write two technical blog posts: one titled Porting VLMs from PyTorch to ONNX in Rust, and one titled Deploying Vision-Language Models on Jetson with kornia-rs. Both will be shared on LinkedIn and developer communities to drive kornia-rs adoption.

After GSoC I intend to remain an active kornia-rs contributor, owning maintenance of the ONNX and TensorRT infrastructure as ORT and Jetson SDK versions evolve. Future work I am interested in includes CoreML support for Apple Silicon and INT8 quantization paths for even smaller edge deployments.

12. Bubbaloop Application: VLM on Physical AI Platform

The kornia-rs GSoC project description is explicit: the expectation is not just a backend, but a real application running on a device. Bubbaloop is the kornia Physical AI platform that ties everything together: it streams multi-camera RTSP feeds, uses Zenoh for pub/sub messaging across devices, and exposes a real-time browser dashboard. This section describes how the ONNX/TensorRT VLM backend I build will be integrated directly into Bubbaloop as a working application with three concrete use cases.

12.1 What Bubbaloop Is

Bubbaloop is an orchestration system for Physical AI deployed on edge hardware. It runs as a set of systemd services: a Zenoh router for low-latency pub/sub messaging, a WebSocket bridge for browser access, and a daemon that manages individual processing nodes. The architecture is designed so that new AI capabilities, such as a VLM inference node, can be plugged in as a new crate under the crates/ directory and wired into the Zenoh message bus. The platform already runs on Linux x86_64 and ARM64, making it a natural fit for Jetson Orin deployment.

12.2 How the VLM Backend Integrates into Bubbaloop

The integration follows Bubbaloop's existing node architecture. A new bubbaloop-vlm node will be added as a Rust crate under crates/. This node subscribes to the camera image topic on the Zenoh bus, passes each frame through the kornia-vlm InferenceBackend (CPU, CUDA, or TensorRT depending on the build target), and publishes the VLM output back to the bus. The browser dashboard receives the output via the WebSocket bridge and displays it in real time alongside the live camera feed.

```
// bubbaloop-vlm node: simplified architecture
async fn run(config: VlmConfig) -> Result<()> {
let session = zenoh::open(config.zenoh).await?;
let cam_sub = session.declare_subscriber("camera/frame").await?;
let vlm_pub = session.declare_publisher("vlm/output").await?;
let backend = OrtBackend::new(&config.model;_path, config.provider)?;
while let Ok(sample) = cam_sub.recv_async().await {
let image = decode_frame(sample.payload())?;
let result = backend.infer(□)?; // kornia-vlm InferenceBackend
vlm_pub.put(result).await?;
}
Ok(())
}
```

12.3 Three Application Use Cases

The Bubbaloop VLM application will demonstrate three real-world use cases, each running on Jetson Orin (or cloud GPU equivalent) with results visible in the Bubbaloop browser dashboard:

Use Case	Input	VLM Task	Bubbaloop Output
Image Captioning	Live camera frame from RTSP stream	Generate natural language description of the scene	Caption overlaid on live feed in browser dashboard
Visual Question Answering (VQA)	Camera frame plus user-typed question via dashboard UI	Answer questions about objects, colors, spatial layout	Answer text published to Zenoh, displayed in dashboard
Scene Understanding	Camera frame from industrial or robotics scenario	Identify objects, estimate layout, flag anomalies	Structured JSON output: object list, positions, alerts

12.4 Application Architecture Diagram

Camera (RTSP) -> kornia-imgproc (resize/normalize) -> kornia-vlm OrtBackend -> Zenoh pub/sub bus -> WebSocket bridge -> Bubbaloop browser dashboard Runtime selection: CPU (development) | ONNX CUDA (GPU server) | TensorRT FP16 (Jetson Orin) All three runtimes use the same bubbaloop-vlm node code. Only the Cargo feature flag changes.

12.5 Deliverable: What Mentors Will See

By the end of Phase 4 (August 8), the following will be demonstrated and recorded:

Deliverable	Format	Timeline
bubbaloop-vlm crate merged into Bubbaloop repo	GitHub PR with passing CI	Phase 4, Week 11
Image captioning running on Jetson or cloud GPU	Demo video: live camera feed with captions in browser	Phase 4, Week 11
VQA demo: user types question, VLM answers in dashboard	Demo video: end-to-end interaction	Phase 4, Week 11
Latency benchmark: camera frame to displayed answer	Criterion results committed to repo	Phase 4, Week 10
Setup guide: run Bubbaloop VLM on Jetson from scratch	Markdown doc in Bubbaloop repo	Phase 5, Week 12

12.6 Updated Timeline with Bubbaloop Integration

The Bubbaloop application work is woven into Phases 3 and 4 of the existing timeline, not added as extra scope. It is the natural endpoint of the backend work: once OrtBackend and TrtBackend are built, the Bubbaloop node is the application layer that uses them.

Phase	Backend Work	Bubbaloop Application Work
Phase 2 (Jun 11-30)	OrtBackend CPU and CUDA complete	Begin bubbaloop-vlm crate skeleton. Zenoh subscriber wired to kornia-vlm.
Phase 3 (Jul 1-18)	Second VLM integrated	Image captioning use case working end-to-end on CPU. Dashboard displays captions.
Phase 4 (Jul 19-Aug 8)	TensorRT backend and Jetson testing	VQA and scene understanding use cases added. Full demo on Jetson. Video recorded.
Phase 5 (Aug 9-18)	Docs and final PR	Bubbaloop setup guide. PR to Bubbaloop repo submitted alongside kornia-rs PR.

13. Why I Am the Right Candidate

13.1 Directly Relevant Technical Experience

I have hands-on experience with the exact technical stack this project requires. I have worked with PyTorch model workflows including forward/backward pass customization, custom module design, and inference optimization. I understand ONNX export including dynamic axes, opset selection, and common failure modes like unsupported ops. I have

used onnxruntime for inference in Python, which directly informs how I will design the Rust OrtBackend. My Kornia contributions involved reading and reasoning about tensor shapes at every stage of a deep learning pipeline, exactly the skill needed to validate ONNX export correctness.

13.2 Evidence from Contributions

Contribution	Relevance to This Project
PR #3565 - LightGlue crash fix (Merged)	Identified a degenerate tensor shape edge case mid-pipeline. Directly analogous to validating ONNX dynamic shape handling.
PR #3520 - FaceDetector Dynamo tests (Merged)	Added torch.compile coverage. torch.compile and ONNX export share similar graph-level constraints.
PR #3614 - Quaternion validation (Pending)	Defensive API design with clear error types. Pattern used throughout the OrtBackend Result-based error handling.
PR #959 - Visdom routing fix (Merged)	Debugged internal request dispatch logic under the hood. Shows ability to navigate unfamiliar codebases effectively.

13.3 Why I Can Execute Independently

I am already in contact with Edgar Riba and Miquel Farre on kornia Discord. I have introduced myself, begun the technical conversation, and understand what the mentors expect. I am not a cold applicant walking in blind.

I am honest about my current Rust level: basic but actively growing. I have studied the ort crate documentation, worked through the kornia-rs codebase to understand its error handling conventions and module structure, and I have a concrete plan for the community bonding period to build proficiency before writing production code. I will not submit code I cannot explain line by line.

I am available full-time during the GSoC coding period with no competing academic commitments. My timezone (UTC+5:30) provides good overlap with European mentors. I am applying to kornia-rs because I want to build something real, in a codebase I respect, with mentors who set a high bar. The ONNX and TensorRT infrastructure will make kornia-vlm genuinely useful for edge deployment and I want to be the person who builds it well.

Thank you for considering this proposal.

Kiran Dharmar - kirandharmar867@gmail.com - github.com/kirandharmar867-maker